CIFP AVILÉS · CENTRO INTEGRADO DE FORMACIÓN PROFESIONAL
Departamento de Informática y Comunicaciones · Principado de Asturias

FP

Solucionario Técnico: Bloque 3

Ecosistema XML: Modelos DOM, Transformaciones XSLT, Parsers SAX/StAX y Mapeo POJO JAXB

MÓDULO PROFESIONAL	CICLO Y UNIDAD	MODALIDAD / ENTORNO	DESTINATARIO / CARÁCTER
Acceso a Datos (0486)	**DAM 2º Curso · UT1**	**Presencial / Distancia · Java 21**	**Docente · Solución Oficial Comentada**

ALINEACIÓN CURRICULAR (RA1: MANEJO DE FICHEROS)

RA1: Manejo de Ficheros | **CE1.c (Recuperar información)** | **CE1.d (Almacenar información)** | **CE1.e (Conversiones entre formatos)** | **CE1.f (Excepciones)**

Módulo: Acceso a Datos | **Ciclo**: DAM2 | **Unidad**: UT1
Hoja de Enunciados: Ver Enunciados para Alumnos
Docente: Sergio Capdevila Díez · CIFP Avilés

Solución Ejercicio 1: Generador de Facturas DOM con Transformer (`FacturaDOM.java`)

```
package soluciones_b3;

import org.w3c.dom.Document;
import org.w3c.dom.Element;

import javax.xml.parsers.DocumentBuilder;
import javax.xml.parsers.DocumentBuilderFactory;
import javax.xml.parsers.ParserConfigurationException;
import javax.xml.transform.OutputKeys;
import javax.xml.transform.Transformer;
import javax.xml.transform.TransformerException;
import javax.xml.transform.TransformerFactory;
import javax.xml.transform.dom.DOMSource;
import javax.xml.transform.stream.StreamResult;
import java.io.File;

public class FacturaDOM {

    public static void main(String[] args) {
        try {
            DocumentBuilderFactory factory = DocumentBuilderFactory.newInstance();
            DocumentBuilder builder = factory.newDocumentBuilder();
```

```
Document doc = builder.newDocument();

// Elemento raíz: <factura>
Element raiz = doc.createElement("factura");
raiz.setAttribute("numero", "FAC-2026-0042");
raiz.setAttribute("fecha", "2026-09-17");
doc.appendChild(raiz);

// <emisor>
Element emisor = doc.createElement("emisor");
emisor.setAttribute("cif", "B-33123456");
Element nomEmisor = doc.createElement("nombre");
nomEmisor.setTextContent("ElectroTech S.L.");
Element ciuEmisor = doc.createElement("ciudad");
ciuEmisor.setTextContent("Avilés");
emisor.appendChild(nomEmisor);
emisor.appendChild(ciuEmisor);
raiz.appendChild(emisor);

// <cliente>
Element cliente = doc.createElement("cliente");
cliente.setAttribute("nif", "71888999X");
Element nomCliente = doc.createElement("nombre");
nomCliente.setTextContent("Desarrollos Astur S.A.");
Element ciuCliente = doc.createElement("ciudad");
ciuCliente.setTextContent("Gijón");
cliente.appendChild(nomCliente);
cliente.appendChild(ciuCliente);
raiz.appendChild(cliente);

// <lineas>
Element lineas = doc.createElement("lineas");
raiz.appendChild(lineas);

// Linea 1
Element l1 = doc.createElement("linea");
l1.setAttribute("id", "1");
Element c1 = doc.createElement("concepto");
c1.setTextContent("Servidor Rack 1U");
Element q1 = doc.createElement("cantidad");
q1.setTextContent("2");
Element p1 = doc.createElement("precioUnitario");
p1.setTextContent("850.00");
l1.appendChild(c1);
l1.appendChild(q1);
l1.appendChild(p1);
lineas.appendChild(l1);

// Linea 2
Element l2 = doc.createElement("linea");
l2.setAttribute("id", "2");
Element c2 = doc.createElement("concepto");
c2.setTextContent("Switch Gigabit 24 Puertos");
Element q2 = doc.createElement("cantidad");
q2.setTextContent("1");
Element p2 = doc.createElement("precioUnitario");
p2.setTextContent("145.50");
l2.appendChild(c2);
l2.appendChild(q2);
l2.appendChild(p2);
```

```
            lineas.appendChild(l2);

            // Volcado a disco con sangría automática
            TransformerFactory tfFactory = TransformerFactory.newInstance();
            Transformer tf = tfFactory.newTransformer();
            tf.setOutputProperty(OutputKeys.INDENT, "yes");
            tf.setOutputProperty("{http://xml.apache.org/xslt}indent-amount", "4");
            tf.setOutputProperty(OutputKeys.ENCODING, "UTF-8");

            File destino = new File("datos/factura_electronica.xml");
            destino.getParentFile().mkdirs();
            tf.transform(new DOMSource(doc), new StreamResult(destino));

            System.out.println("✅ Factura XML generada e indentada en: " +
destino.getPath());

        } catch (ParserConfigurationException | TransformerException e) {
            System.err.println("Error generando factura DOM: " + e.getMessage());
        }
    }
}
```

▶️ Salida real (Java 21)

Consola: `✅ Factura XML generada e indentada en: datos/factura_electronica.xml`

`datos/factura_electronica.xml`:

```
<?xml version="1.0" encoding="UTF-8" standalone="no"?>
<factura fecha="2026-09-17" numero="FAC-2026-0042">
    <emisor cif="B-33123456">
        <nombre>ElectroTech S.L.</nombre>
        <ciudad>Avilés</ciudad>
    </emisor>
    <cliente nif="71888999X">
        <nombre>Desarrollos Astur S.A.</nombre>
        <ciudad>Gijón</ciudad>
    </cliente>
    <lineas>
        <linea id="1">
            <concepto>Servidor Rack 1U</concepto>
            <cantidad>2</cantidad>
            <precioUnitario>850.00</precioUnitario>
        </linea>
        <linea id="2">
            <concepto>Switch Gigabit 24 Puertos</concepto>
            <cantidad>1</cantidad>
            <precioUnitario>145.50</precioUnitario>
        </linea>
    </lineas>
</factura>
```

> **Nota:** el `Transformer` del JDK escribe los atributos en orden alfabético (`fecha` antes que `numero`); el documento es equivalente.

Solución Ejercicio 2: Modificador de Tarifas DOM In-Situ (`GestorTarifasDOM.java`)

```
package soluciones_b3;

import org.w3c.dom.Document;
import org.w3c.dom.Element;
import org.w3c.dom.Node;
import org.w3c.dom.NodeList;

import javax.xml.parsers.DocumentBuilderFactory;
import javax.xml.parsers.ParserConfigurationException;
import javax.xml.transform.OutputKeys;
import javax.xml.transform.Transformer;
import javax.xml.transform.TransformerException;
import javax.xml.transform.TransformerFactory;
import javax.xml.transform.dom.DOMSource;
import javax.xml.transform.stream.StreamResult;
import org.xml.sax.SAXException;

import java.io.File;
import java.io.IOException;
import java.util.Locale;

public class GestorTarifasDOM {

    public static void aplicarDescuentoCategoria(File archivo, String categoriaObjetivo,
double porcentaje) {
        try {
            Document doc = DocumentBuilderFactory.newInstance()
                    .newDocumentBuilder()
                    .parse(archivo);

            // Junta textos contiguos (no elimina los #text de sangría)
            doc.getDocumentElement().normalize();

            // Recorremos los hijos de <catalogo>: entre ellos están los #text de los
saltos de línea
            NodeList productos = doc.getDocumentElement().getChildNodes();
            int modificados = 0;


System.out.println("======================================================================");
            System.out.printf("   APLICANDO %.1f%% DE DESCUENTO A CATEGORÍA: %s%n",
porcentaje, categoriaObjetivo);

System.out.println("======================================================================");

            for (int i = 0; i < productos.getLength(); i++) {
                Node nodo = productos.item(i);
```

```
                if (nodo.getNodeType() != Node.ELEMENT_NODE) continue;   // salta los
#text (si no, ClassCastException)

                Element elemProducto = (Element) nodo;
                if (!elemProducto.getTagName().equals("producto")) continue;
                String cat = elemProducto.getAttribute("categoria");

                if (categoriaObjetivo.equalsIgnoreCase(cat)) {
                    NodeList hijos = elemProducto.getElementsByTagName("precio");
                    if (hijos.getLength() > 0) {
                        Element elemPrecio = (Element) hijos.item(0);
                        double precioAntiguo =
Double.parseDouble(elemPrecio.getTextContent().trim());
                        double nuevoPrecio = precioAntiguo * (1.0 - (porcentaje /
100.0));

                        elemPrecio.setTextContent(String.format(Locale.US, "%.2f",
nuevoPrecio));

                        String nombre =
elemProducto.getElementsByTagName("nombre").item(0).getTextContent();
                        System.out.printf("  - %-28s : %6.2f € -> %6.2f €%n", nombre,
precioAntiguo, nuevoPrecio);
                        modificados++;
                    }
                }
            }

            // Quitar la sangría vieja: el Transformer genera la suya (si no, líneas en
blanco en cada ejecución)
            eliminarNodosEnBlanco(doc.getDocumentElement());

            // Volcar de nuevo al archivo con sangría de 4 espacios
            Transformer tf = TransformerFactory.newInstance().newTransformer();
            tf.setOutputProperty(OutputKeys.INDENT, "yes");
            tf.setOutputProperty(OutputKeys.ENCODING, "UTF-8");
            tf.setOutputProperty("{http://xml.apache.org/xslt}indent-amount", "4");
            tf.transform(new DOMSource(doc), new StreamResult(archivo));

            System.out.println("-------------------------------------------------------
---------");
            System.out.printf("✅ Tarifa actualizada en disco: %d productos
modificados.%n", modificados);

        } catch (ParserConfigurationException | SAXException | IOException |
TransformerException e) {
            System.err.println("Error actualizando tarifas DOM: " + e.getMessage());
        }
    }

    /** Elimina recursivamente los nodos #text que solo contienen espacios o saltos de
línea. */
    private static void eliminarNodosEnBlanco(Node nodo) {
        NodeList hijos = nodo.getChildNodes();
        for (int i = hijos.getLength() - 1; i >= 0; i--) {       // de atrás adelante:
al borrar se desplazan
            Node hijo = hijos.item(i);
            if (hijo.getNodeType() == Node.TEXT_NODE && hijo.getTextContent().isBlank())
{
                nodo.removeChild(hijo);
```

```
            } else if (hijo.getNodeType() == Node.ELEMENT_NODE) {
                eliminarNodosEnBlanco(hijo);
            }
        }
    }

    public static void main(String[] args) {
        File f = new File("datos/catalogo_tienda.xml");
        aplicarDescuentoCategoria(f, "Perifericos", 15.0);
    }
}
```

▶️ Salida real (Java 21, primera ejecución)

```
================================================================
   APLICANDO 15,0% DE DESCUENTO A CATEGORÍA: Perifericos
================================================================
    - Raton Ergonomico Inalambrico :  34,50 € ->  29,33 €
    - Teclado Mecanico RGB         :  65,00 € ->  55,25 €
----------------------------------------------------------------
✅ Tarifa actualizada en disco: 2 productos modificados.
```

`datos/catalogo_tienda.xml` tras la ejecución (sin líneas en blanco; una segunda ejecución da 29,33 → 24,93 y 55,25 → 46,96 y el fichero sigue igual de limpio):

```
<?xml version="1.0" encoding="UTF-8" standalone="no"?>
<catalogo>
    <producto categoria="Hardware" id="P01">
        <nombre>Disco SSD NVMe 1TB</nombre>
        <precio>89.95</precio>
        <stock>25</stock>
    </producto>
    <producto categoria="Perifericos" id="P02">
        <nombre>Raton Ergonomico Inalambrico</nombre>
        <precio>29.33</precio>
        <stock>12</stock>
    </producto>
    <producto categoria="Perifericos" id="P03">
        <nombre>Teclado Mecanico RGB</nombre>
        <precio>55.25</precio>
        <stock>8</stock>
    </producto>
</catalogo>
```

> **Notas para el aula:** se recorren los hijos de `<catalogo>` con `getChildNodes()`, así que el filtro `ELEMENT_NODE` es imprescindible (el primer hijo es un `#text`). Sin `eliminarNodosEnBlanco()`, el árbol conserva la sangría original y `INDENT=yes` añade la suya: aparecen líneas en blanco que crecen en cada ejecución (apartado 5.3).

Solución Ejercicio 3: Analizador Dual SAX y StAX (`AnalizadorDualXML.java`)

```
package soluciones_b3;

import org.xml.sax.Attributes;
import org.xml.sax.SAXException;
import org.xml.sax.helpers.DefaultHandler;

import javax.xml.parsers.ParserConfigurationException;
import javax.xml.parsers.SAXParserFactory;
import javax.xml.stream.XMLInputFactory;
import javax.xml.stream.XMLStreamConstants;
import javax.xml.stream.XMLStreamException;
import javax.xml.stream.XMLStreamReader;
import java.io.File;
import java.io.FileInputStream;
import java.io.IOException;
import java.io.InputStream;
import java.util.Locale;

public class AnalizadorDualXML {

    // ================================================================================
    // PARTE A: SAX PUSH (CatalogoStatsHandler)
    // ================================================================================
    public static class CatalogoStatsHandler extends DefaultHandler {
        private final StringBuilder buffer = new StringBuilder();
        private int totalProductos = 0;
        private double valorTotalInventario = 0;
        private double precioActual = 0;
        private int stockActual = 0;

        private String nombreMasCaro = "";
        private double precioMasCaro = -1;
        private String nombreActual = "";

        @Override
        public void startElement(String uri, String localName, String qName, Attributes
attributes) {
            buffer.setLength(0); // Limpiar buffer al abrir cualquier etiqueta
            if (qName.equals("producto")) {          // qName: el localName llega vacío
                totalProductos++;
            }
        }

        @Override
        public void characters(char[] ch, int start, int length) {
            buffer.append(ch, start, length); // Acumulación estricta
        }

        @Override
        public void endElement(String uri, String localName, String qName) {
            String texto = buffer.toString().trim();
            if (qName.equals("nombre")) {
                nombreActual = texto;
            } else if (qName.equals("precio")) {
```

```
                precioActual = Double.parseDouble(texto);
                if (precioActual > precioMasCaro) {
                    precioMasCaro = precioActual;
                    nombreMasCaro = nombreActual;
                }
            } else if (qName.equals("stock")) {
                stockActual = Integer.parseInt(texto);
                valorTotalInventario += (precioActual * stockActual);
            }
        }

        @Override
        public void endDocument() {

System.out.println("=============================================================================");
            System.out.println("     INFORME ESTADÍSTICO SAX (MEMORIA MÍNIMA Y
CONSTANTE)      ");

System.out.println("=============================================================================");
            System.out.printf(Locale.US, "1. Total productos analizados: %d%n",
totalProductos);
            System.out.printf(Locale.US, "2. Valor total del inventario: %,.2f €%n",
valorTotalInventario);
            System.out.printf(Locale.US, "3. Producto más caro: %s (%.2f €)%n",
nombreMasCaro, precioMasCaro);

System.out.println("=============================================================================");
        }
    }

    public static void ejecutarSAX(File archivo) {
        try {
            SAXParserFactory.newInstance().newSAXParser().parse(archivo, new
CatalogoStatsHandler());
        } catch (ParserConfigurationException | SAXException | IOException e) {
            System.err.println("Error en parseo SAX: " + e.getMessage());
        }
    }

    // ============================================================================
    // PARTE B: StAX PULL (BuscadorStAX con early exit)
    // ============================================================================
    public static void buscarProductoPorId(File archivo, String idBuscado) {
        System.out.println("\n--- Búsqueda Directa con StAX Cursor (Early Exit) ---");
        try (InputStream in = new FileInputStream(archivo)) {        // bytes: StAX lee
la codificación del prólogo
            XMLStreamReader reader =
XMLInputFactory.newInstance().createXMLStreamReader(in);
            try {
                boolean enProductoObjetivo = false;
                String etiqueta = "";                // última etiqueta abierta
                String nombre = "";
                String precio = "";
                String stock = "";

                while (reader.hasNext()) {
                    int evento = reader.next();

                    if (evento == XMLStreamConstants.START_ELEMENT) {
                        etiqueta = reader.getLocalName();
```

```
                        if ("producto".equals(etiqueta)) {
                            enProductoObjetivo =
idBuscado.equals(reader.getAttributeValue(null, "id"));
                        }
                    } else if (evento == XMLStreamConstants.CHARACTERS
                               && enProductoObjetivo && !reader.isWhiteSpace()) {
                        // Texto del producto buscado (los saltos de línea de la sangría
se descartan)
                        if ("nombre".equals(etiqueta)) nombre = reader.getText();
                        else if ("precio".equals(etiqueta)) precio = reader.getText();
                        else if ("stock".equals(etiqueta)) stock = reader.getText();
                    } else if (evento == XMLStreamConstants.END_ELEMENT) {
                        if ("producto".equals(reader.getLocalName()) &&
enProductoObjetivo) {
                            System.out.printf("🎯 Encontrado con StAX: [%s] %s | Precio:
%s € | Stock: %s uds.%n",
                                    idBuscado, nombre, precio, stock);
                            // Demostración de ventaja StAX: Detenemos la lectura al
instante
                            System.out.println("⚡ Bucle StAX detenido con 'break' sin
leer el resto del archivo.");
                            break;
                        }
                    }
                }
            } finally {
                reader.close();   // XMLStreamReader no es AutoCloseable
            }
        } catch (IOException | XMLStreamException e) {
            System.err.println("Error en búsqueda StAX: " + e.getMessage());
        }
    }

    public static void main(String[] args) {
        File f = new File("datos/catalogo_tienda.xml");
        ejecutarSAX(f);
        buscarProductoPorId(f, "P02");
    }
}
```

▶️ Salida real (Java 21, sobre el `catalogo_tienda.xml` original)

```
================================================================
     INFORME ESTADÍSTICO SAX (MEMORIA MÍNIMA Y CONSTANTE)
================================================================

1. Total productos analizados: 3
2. Valor total del inventario: 3,182.75 €
3. Producto más caro: Disco SSD NVMe 1TB (89.95 €)
================================================================

--- Búsqueda Directa con StAX Cursor (Early Exit) ---
🎯 Encontrado con StAX: [P02] Raton Ergonomico Inalambrico | Precio: 34.50 € | Stock: 12
uds.
⚡ Bucle StAX detenido con 'break' sin leer el resto del archivo.
```

> **Notas para el aula:** en SAX se compara con `qName` y `equals()` (el `localName` llega vacío con la factoría por defecto; XML distingue mayúsculas). `XMLStreamReader` no es `AutoCloseable`: se cierra en un `finally`. El lector StAX se crea sobre un `FileInputStream` (bytes) para que respete la codificación del prólogo. El texto se toma de los eventos `CHARACTERS` filtrando `isWhiteSpace()`, como en `BuscarStAX.java` (§5.5); `getElementText()` también serviría, pero no se ha visto en clase. Con textos muy largos o entidades (`&`), un mismo texto puede llegar en varios `CHARACTERS`: entonces habría que acumularlo como en SAX.

Solución Ejercicio 4: Mapeo Declarativo Objeto-XML con JAXB (`BibliotecaJAXB.java`)

```
package soluciones_b3;

import jakarta.xml.bind.JAXBContext;
import jakarta.xml.bind.JAXBException;
import jakarta.xml.bind.Marshaller;
import jakarta.xml.bind.Unmarshaller;
import jakarta.xml.bind.annotation.*;

import java.io.File;
import java.util.ArrayList;
import java.util.List;

public class BibliotecaJAXB {

    // 1. Clase Libro con atributos y elementos
    @XmlAccessorType(XmlAccessType.FIELD)
    public static class Libro {
        @XmlAttribute
        private String isbn;

        @XmlElement
        private String titulo;

        @XmlElement
        private String autor;

        @XmlElement
        private double precio;

        @XmlElement
        private int paginas;

        public Libro() {} // Constructor sin argumentos obligatorio para JAXB

        public Libro(String isbn, String titulo, String autor, double precio, int
paginas) {
            this.isbn = isbn;
            this.titulo = titulo;
            this.autor = autor;
            this.precio = precio;
            this.paginas = paginas;
```

```
        }

        @Override
        public String toString() {
            return String.format("  * [%s] '%s' de %s (%d págs.) -> %.2f €",
                    isbn, titulo, autor, paginas, precio);
        }
    }

    // 2. Clase Biblioteca (Raíz)
    @XmlRootElement(name = "biblioteca")
    @XmlAccessorType(XmlAccessType.FIELD)
    public static class Biblioteca {
        @XmlAttribute
        private String nombre;

        @XmlElement(name = "libro")      // cada libro es un <libro> hijo directo de
<biblioteca>
        private List<Libro> libros = new ArrayList<>();

        public Biblioteca() {}

        public Biblioteca(String nombre) {
            this.nombre = nombre;
        }

        public void agregarLibro(Libro l) {
            this.libros.add(l);
        }

        public List<Libro> getLibros() {
            return libros;
        }
    }

    // 3. Operaciones Marshaller / Unmarshaller
    public static void exportarBiblioteca(Biblioteca b, File destino) throws
JAXBException {
        JAXBContext context = JAXBContext.newInstance(Biblioteca.class);
        Marshaller m = context.createMarshaller();
        m.setProperty(Marshaller.JAXB_FORMATTED_OUTPUT, true);
        m.setProperty(Marshaller.JAXB_ENCODING, "UTF-8");

        destino.getParentFile().mkdirs();
        m.marshal(b, destino);
        System.out.println("✅ Biblioteca serializada a XML con JAXB en: " +
destino.getPath());
    }

    public static Biblioteca importarBiblioteca(File origen) throws JAXBException {
        JAXBContext context = JAXBContext.newInstance(Biblioteca.class);
        Unmarshaller um = context.createUnmarshaller();
        return (Biblioteca) um.unmarshal(origen);
    }

    public static void main(String[] args) throws JAXBException {
        File f = new File("datos/biblioteca.xml");

        Biblioteca bib = new Biblioteca("Biblioteca Municipal de Avilés");
        bib.agregarLibro(new Libro("978-84-1", "Aprende Java 21 en FP", "Sergio
```

```
Capdevila", 29.95, 450));
        bib.agregarLibro(new Libro("978-84-2", "Patrones de Acceso a Datos", "Martin
Fowler", 42.50, 520));
        bib.agregarLibro(new Libro("978-84-3", "Arquitectura Limpia", "Robert C.
Martin", 38.00, 380));

        // 1. Guardar en XML
        exportarBiblioteca(bib, f);

        // 2. Cargar desde XML
        System.out.println("\n--- Recuperando desde XML con JAXB ---");
        Biblioteca recuperada = importarBiblioteca(f);
        System.out.println("Biblioteca cargada: " + recuperada.nombre);
        recuperada.getLibros().forEach(System.out::println);
    }
}
```

▶️ Comprobación

Compilado con Java 21 contra la API de Jakarta XML Binding (no ejecutado aquí: necesita `jaxb-runtime` del `pom.xml`). Con el runtime, `datos/biblioteca.xml` contiene `<biblioteca nombre="Biblioteca Municipal de Avilés">` con tres `<libro isbn="…">` hijos directos y la consola lista los tres libros recuperados.

> **Notas para el aula:** la versión anterior no compilaba (`forEach(System.out.println)` → `forEach(System.out::println)`). Si se quisiera un `<libros>` que envolviera la lista, se añadiría `@XmlElementWrapper(name = "libros")` sobre el campo. Si falta un constructor sin argumentos en `Libro` o `Biblioteca`, `JAXBContext.newInstance()` lanza `JAXBException`.

Solución Ejercicio 5: Lector del Horario del Grupo con DOM (`HorarioDOM.java`)

```
package soluciones_b3;

import org.w3c.dom.Document;
import org.w3c.dom.Element;
import org.w3c.dom.Node;
import org.w3c.dom.NodeList;
import org.xml.sax.SAXException;

import javax.xml.parsers.DocumentBuilderFactory;
import javax.xml.parsers.ParserConfigurationException;
import java.io.File;
import java.io.IOException;
import java.util.Map;
import java.util.TreeMap;

public class HorarioDOM {

    public static void main(String[] args) {
        try {
            Document doc = DocumentBuilderFactory.newInstance()
                    .newDocumentBuilder()
```

```
                    .parse(new File("datos/horario_2DAM.xml"));
            doc.getDocumentElement().normalize();

            Element horario = doc.getDocumentElement();
            String aulaBase = horario.getAttribute("aulaBase");
            System.out.printf("Horario del grupo %s (curso %s)%n",
                    horario.getAttribute("grupo"), horario.getAttribute("curso"));

            // 1. Qué hay realmente entre los hijos de <horario>
            NodeList hijos = horario.getChildNodes();
            int elementos = 0, textos = 0, otros = 0;
            for (int i = 0; i < hijos.getLength(); i++) {
                short tipo = hijos.item(i).getNodeType();
                if (tipo == Node.ELEMENT_NODE) elementos++;
                else if (tipo == Node.TEXT_NODE) textos++;
                else otros++;                                   // el comentario <!-- ...
-->
            }
            System.out.printf("Hijos de <horario>: %d (%d elementos, %d #text, %d
comentario)%n",
                    hijos.getLength(), elementos, textos, otros);

            // 2. Recorrido de dos niveles con getChildNodes(): el filtro ELEMENT_NODE
es obligatorio
            Map<String, Integer> horasPorModulo = new TreeMap<>();
            for (int i = 0; i < hijos.getLength(); i++) {
                Node nDia = hijos.item(i);
                if (nDia.getNodeType() != Node.ELEMENT_NODE) continue;
                Element dia = (Element) nDia;
                System.out.println(dia.getAttribute("nombre") + ":");

                NodeList sesiones = dia.getChildNodes();
                for (int j = 0; j < sesiones.getLength(); j++) {
                    Node nSesion = sesiones.item(j);
                    if (nSesion.getNodeType() != Node.ELEMENT_NODE) continue;   // #text
y #comment fuera
                    Element sesion = (Element) nSesion;

                    String modulo = sesion.getTextContent().trim();
                    String aula = sesion.getAttribute("aula");                // "" si
el atributo no existe
                    if (aula.isEmpty()) aula = aulaBase;

                    System.out.printf("  %sª hora  %-5s %s%n",
sesion.getAttribute("hora"), modulo, aula);
                    horasPorModulo.put(modulo, horasPorModulo.getOrDefault(modulo, 0) +
1);
                }
            }

            System.out.println("Horas semanales por módulo: " + horasPorModulo);

        } catch (ParserConfigurationException | SAXException | IOException e) {
            System.err.println("Error leyendo el horario: " + e.getMessage());
        }
    }
}
```

▶️ Salida real (Java 21)

```
Horario del grupo 2DAM (curso 2026-2027)
Hijos de <horario>: 9 (3 elementos, 5 #text, 1 comentario)
Lunes:
  1ª hora  AD    A12
  2ª hora  AD    A12
  3ª hora  DI    LAB2
Martes:
  1ª hora  PMDM  A12
  2ª hora  PSP   A12
  3ª hora  SGE   LAB1
Miércoles:
  1ª hora  AD    A12
  2ª hora  DI    LAB2
  3ª hora  PMDM  A12
Horas semanales por módulo: {AD=3, DI=2, PMDM=2, PSP=1, SGE=1}
```

> **Notas para el aula:** los 9 hijos de `<horario>` son los 3 `<dia>`, el comentario y 5 `#text` (la sangría antes y después de cada uno). Dentro del martes, el comentario `<!-- Recreo... -->` es un hijo más de `<dia>`: un filtro mal escrito como `getNodeType() != Node.TEXT_NODE` dejaría pasar el comentario y el *cast* a `Element` lanzaría `ClassCastException`. Por eso se pregunta por lo que se quiere (`ELEMENT_NODE`) y no por lo que se quiere descartar. `getAttribute()` devuelve `""` cuando el atributo no existe, nunca `null`.

Solución Ejercicio 6: Conversor de Catálogo de Cursos de CSV a XML (`CursosCsvAXml.java`)

```
package soluciones_b3;

import org.w3c.dom.Document;
import org.w3c.dom.Element;

import javax.xml.parsers.DocumentBuilderFactory;
import javax.xml.parsers.ParserConfigurationException;
import javax.xml.transform.OutputKeys;
import javax.xml.transform.Transformer;
import javax.xml.transform.TransformerException;
import javax.xml.transform.TransformerFactory;
import javax.xml.transform.dom.DOMSource;
import javax.xml.transform.stream.StreamResult;
import java.io.BufferedReader;
import java.io.File;
import java.io.IOException;
import java.nio.charset.StandardCharsets;
import java.nio.file.Files;
import java.nio.file.Path;
import java.util.Locale;
import java.util.Map;
```

```
import java.util.TreeMap;

public class CursosCsvAXml {

    /** Crea <etiqueta>texto</etiqueta> y lo engancha a 'padre'. */
    static Element hijo(Document doc, Element padre, String etiqueta, String texto) {
        Element e = doc.createElement(etiqueta);
        e.setTextContent(texto);
        padre.appendChild(e);
        return e;
    }

    public static void main(String[] args) {
        Path csv = Path.of("datos", "cursos.csv");
        File xml = new File("datos/cursos.xml");

        try (BufferedReader br = Files.newBufferedReader(csv, StandardCharsets.UTF_8)) {
            Document doc =
DocumentBuilderFactory.newInstance().newDocumentBuilder().newDocument();
            Element raiz = doc.createElement("catalogoCursos");
            raiz.setAttribute("origen", "cursos.csv");
            doc.appendChild(raiz);

            // Un <familia> por familia profesional, en orden alfabético (TreeMap)
            Map<String, Element> familias = new TreeMap<>();
            int convertidos = 0, numLinea = 1;
            br.readLine();                                          // cabecera

            String linea;
            while ((linea = br.readLine()) != null) {
                numLinea++;
                String[] c = linea.split(";", -1);
                if (c.length != 6) {
                    System.out.printf("⚠️ Línea %d descartada (%d campos): %s%n",
numLinea, c.length, linea);
                    continue;
                }
                int horas, plazas;
                double precio;
                try {
                    horas = Integer.parseInt(c[3].trim());
                    precio = Double.parseDouble(c[4].trim());
                    plazas = Integer.parseInt(c[5].trim());
                } catch (NumberFormatException e) {
                    System.out.printf("⚠️ Línea %d descartada (número no válido): %s%n",
numLinea, linea);
                    continue;
                }

                Element familia = familias.get(c[2]);
                if (familia == null) {                              // primera vez que
sale esta familia
                    familia = doc.createElement("familia");
                    familia.setAttribute("nombre", c[2]);
                    familias.put(c[2], familia);
                }

                Element curso = doc.createElement("curso");
                curso.setAttribute("codigo", c[0]);
                curso.setAttribute("plazas", String.valueOf(plazas));
```

```
                hijo(doc, curso, "titulo", c[1]);                  // el '&' lo escapa
DOM: &amp;
                hijo(doc, curso, "horas", String.valueOf(horas));
                Element ePrecio = hijo(doc, curso, "precio", String.format(Locale.US,
"%.2f", precio));
                ePrecio.setAttribute("moneda", "EUR");
                familia.appendChild(curso);
                convertidos++;
            }

            for (Element familia : familias.values()) {
                raiz.appendChild(familia);
            }
            raiz.setAttribute("total", String.valueOf(convertidos));   // el árbol se
puede retocar al final

            Transformer tf = TransformerFactory.newInstance().newTransformer();
            tf.setOutputProperty(OutputKeys.INDENT, "yes");
            tf.setOutputProperty(OutputKeys.ENCODING, "UTF-8");
            tf.setOutputProperty("{http://xml.apache.org/xslt}indent-amount", "4");
            tf.transform(new DOMSource(doc), new StreamResult(xml));

            System.out.printf("✅ %s generado: %d cursos en %d familias.%n",
xml.getPath(), convertidos, familias.size());

        } catch (IOException e) {
            System.err.println("Error leyendo el CSV: " + e.getMessage());
        } catch (ParserConfigurationException | TransformerException e) {
            System.err.println("Error generando el XML: " + e.getMessage());
        }
    }
}
```

▶️ Salida real (Java 21)

Consola:

```
⚠️ Línea 5 descartada (número no válido): IFC03;Docker y
contenedores;Informática;cuarenta;150.00;12
⚠️ Línea 6 descartada (5 campos): ELE01;Domótica KNX;Electricidad;50;210.00
✅ datos/cursos.xml generado: 4 cursos en 3 familias.
```

`datos/cursos.xml`:

```
<?xml version="1.0" encoding="UTF-8" standalone="no"?>
<catalogoCursos origen="cursos.csv" total="4">
    <familia nombre="Administración">
        <curso codigo="ADG01" plazas="25">
            <titulo>Contabilidad con hoja de cálculo</titulo>
            <horas>30</horas>
            <precio moneda="EUR">95.00</precio>
        </curso>
    </familia>
    <familia nombre="Comercio">
        <curso codigo="COM01" plazas="30">
            <titulo>Atención al cliente &amp;amp; redes sociales</titulo>
```

```
            <horas>20</horas>
            <precio moneda="EUR">60.00</precio>
        </curso>
    </familia>
    <familia nombre="Informática">
        <curso codigo="IFC01" plazas="20">
            <titulo>Python para análisis de datos</titulo>
            <horas>60</horas>
            <precio moneda="EUR">180.00</precio>
        </curso>
        <curso codigo="IFC02" plazas="15">
            <titulo>Ciberseguridad básica</titulo>
            <horas>40</horas>
            <precio moneda="EUR">120.50</precio>
        </curso>
    </familia>
</catalogoCursos>
```

Notas para el aula: es una conversión entre formatos (CE1.e). El `TreeMap<String, Element>` hace dos trabajos: encuentra el `<familia>` ya creado y los devuelve en orden alfabético con `values()`. `ADG01` tiene precio `95` y sale `95.00` gracias al `%.2f` con `Locale.US`. Comprueba en el XML que `&` se ha escrito como `&`: si se generara el XML concatenando cadenas, ese `&` daría un fichero mal formado que ningún parser aceptaría. Los atributos salen en orden alfabético (`codigo` antes que `plazas`; `origen` antes que `total`).

Solución Ejercicio 7: Revisión de Caducidades del Botiquín con DOM y SAX (`BotiquinXML.java`)

```
package soluciones_b3;

import org.w3c.dom.Document;
import org.w3c.dom.Element;
import org.w3c.dom.Node;
import org.w3c.dom.NodeList;
import org.xml.sax.Attributes;
import org.xml.sax.SAXException;
import org.xml.sax.helpers.DefaultHandler;

import javax.xml.parsers.DocumentBuilderFactory;
import javax.xml.parsers.ParserConfigurationException;
import javax.xml.parsers.SAXParserFactory;
import javax.xml.transform.OutputKeys;
import javax.xml.transform.Transformer;
import javax.xml.transform.TransformerException;
import javax.xml.transform.TransformerFactory;
import javax.xml.transform.dom.DOMSource;
import javax.xml.transform.stream.StreamResult;
import java.io.File;
import java.io.IOException;
import java.time.LocalDate;
```

```
import java.util.ArrayList;
import java.util.List;

public class BotiquinXML {

    // ========================================================================
    // PARTE A: DOM — retirar lotes caducados con removeChild() y guardar
    // ========================================================================
    public static void retirarCaducados(File origen, File destino, LocalDate corte) {
        try {
            Document doc =
DocumentBuilderFactory.newInstance().newDocumentBuilder().parse(origen);
            doc.getDocumentElement().normalize();

            Element raiz = doc.getDocumentElement();
            NodeList medicamentos = raiz.getElementsByTagName("medicamento");
            List<Element> sinLotes = new ArrayList<>();
            int retirados = 0;

            for (int i = 0; i < medicamentos.getLength(); i++) {
                Element med = (Element) medicamentos.item(i);
                // getElementsByTagName() devuelve una lista "viva": al quitar un nodo,
los siguientes
                // se desplazan una posición. Por eso se recorre de atrás adelante (como
eliminarNodosEnBlanco()).
                NodeList lotes = med.getElementsByTagName("lote");
                for (int j = lotes.getLength() - 1; j >= 0; j--) {
                    Element lote = (Element) lotes.item(j);
                    LocalDate caducidad =
LocalDate.parse(lote.getAttribute("caducidad"));
                    if (caducidad.isBefore(corte)) {
                        System.out.printf("  🗑️  Lote %s de %s (caducó el %s, %s
uds.)%n", lote.getAttribute("numero"),
                                med.getAttribute("nombre"), caducidad,
lote.getTextContent().trim());
                        med.removeChild(lote);                  // el padre quita a su
hijo
                        retirados++;
                    }
                }
                if (lotes.getLength() == 0) {                   // la lista viva ya
refleja los borrados
                    sinLotes.add(med);                          // no se borra aquí:
desplazaría 'medicamentos'
                }
            }
            for (Element med : sinLotes) {
                System.out.println("  🗑️  Medicamento sin lotes: " +
med.getAttribute("nombre"));
                raiz.removeChild(med);
            }

            eliminarNodosEnBlanco(doc.getDocumentElement());     // si no, quedan líneas
en blanco

            Transformer tf = TransformerFactory.newInstance().newTransformer();
            tf.setOutputProperty(OutputKeys.INDENT, "yes");
            tf.setOutputProperty(OutputKeys.ENCODING, "UTF-8");
            tf.setOutputProperty("{http://xml.apache.org/xslt}indent-amount", "4");
            tf.transform(new DOMSource(doc), new StreamResult(destino));
```

```
            System.out.printf("✅ %d lotes retirados. Guardado en %s%n", retirados,
destino.getPath());

        } catch (ParserConfigurationException | SAXException | IOException |
TransformerException e) {
            System.err.println("Error en la revisión DOM: " + e.getMessage());
        }
    }

    /** Elimina recursivamente los nodos #text que solo contienen espacios o saltos de
línea (apartado 5.3). */
    static void eliminarNodosEnBlanco(Node nodo) {
        NodeList hijos = nodo.getChildNodes();
        for (int i = hijos.getLength() - 1; i >= 0; i--) {
            Node hijo = hijos.item(i);
            if (hijo.getNodeType() == Node.TEXT_NODE && hijo.getTextContent().isBlank())
{
                nodo.removeChild(hijo);
            } else if (hijo.getNodeType() == Node.ELEMENT_NODE) {
                eliminarNodosEnBlanco(hijo);
            }
        }
    }

    // ================================================================================
    // PARTE B: SAX — resumen del fichero revisado (atributos + texto acumulado)
    // ================================================================================
    public static class ResumenBotiquinHandler extends DefaultHandler {
        private final StringBuilder buffer = new StringBuilder();
        private final LocalDate avisoHasta;                 // lotes que caducan antes
de esta fecha
        private String medicamento = "";
        private String numeroLote = "";
        private String caducidadLote = "";
        private int lotesMed = 0, unidadesMed = 0;
        private int totalMed = 0, totalLotes = 0, totalUnidades = 0;
        private final StringBuilder avisos = new StringBuilder();

        public ResumenBotiquinHandler(LocalDate avisoHasta) {
            this.avisoHasta = avisoHasta;
        }

        @Override
        public void startElement(String uri, String localName, String qName, Attributes
atts) {
            buffer.setLength(0);
            if (qName.equals("medicamento")) {
                medicamento = atts.getValue("nombre");          // los atributos llegan
en startElement
                lotesMed = 0;
                unidadesMed = 0;
            } else if (qName.equals("lote")) {
                numeroLote = atts.getValue("numero");
                caducidadLote = atts.getValue("caducidad");
            }
        }

        @Override
        public void characters(char[] ch, int start, int length) {
            buffer.append(ch, start, length);                   // el texto puede
```

```
llegar troceado
        }

        @Override
        public void endElement(String uri, String localName, String qName) {
            if (qName.equals("lote")) {
                int unidades = Integer.parseInt(buffer.toString().trim());
                lotesMed++;
                unidadesMed += unidades;
                if (LocalDate.parse(caducidadLote).isBefore(avisoHasta)) {
                    avisos.append(String.format("  ⚠️ %s (%s) caduca el %s%n",
numeroLote, medicamento, caducidadLote));
                }
            } else if (qName.equals("medicamento")) {
                System.out.printf("  %-24s %d lote(s) %4d uds.%n", medicamento,
lotesMed, unidadesMed);
                totalMed++;
                totalLotes += lotesMed;
                totalUnidades += unidadesMed;
            }
        }

        @Override
        public void endDocument() {
            System.out.printf("Total: %d medicamentos, %d lotes, %d unidades%n",
totalMed, totalLotes, totalUnidades);
            System.out.print(avisos.length() > 0 ? "Caducan en los próximos 30 días:\n"
+ avisos : "Sin caducidades próximas.\n");
        }
    }

    public static void main(String[] args) {
        File original = new File("datos/botiquin.xml");
        File revisado = new File("datos/botiquin_revisado.xml");
        LocalDate corte = LocalDate.parse("2026-10-05");        // fecha de la revisión
(fija para poder comprobar)

        System.out.println("=== Revisión DOM del botiquín a fecha " + corte + " ===");
        retirarCaducados(original, revisado, corte);

        System.out.println("=== Resumen SAX de " + revisado.getName() + " ===");
        try {
            SAXParserFactory.newInstance().newSAXParser()
                    .parse(revisado, new ResumenBotiquinHandler(corte.plusDays(30)));
        } catch (ParserConfigurationException | SAXException | IOException e) {
            System.err.println("Error en el resumen SAX: " + e.getMessage());
        }
    }
}
```

▶️ Salida real (Java 21)

Consola:

```
=== Revisión DOM del botiquín a fecha 2026-10-05 ===
  🗑️  Lote L2301 de Paracetamol 500 mg (caducó el 2026-03-31, 20 uds.)
  🗑️  Lote L2210 de Ibuprofeno 400 mg (caducó el 2025-12-31, 10 uds.)
  🗑️  Lote L2502 de Suero fisiológico 5 ml (caducó el 2026-09-30, 15 uds.)
  🗑️  Lote L2501 de Suero fisiológico 5 ml (caducó el 2026-10-01, 40 uds.)
  🗑️  Medicamento sin lotes: Ibuprofeno 400 mg
✅ 4 lotes retirados. Guardado en datos/botiquin_revisado.xml
=== Resumen SAX de botiquin_revisado.xml ===
  Paracetamol 500 mg       1 lote(s)   30 uds.
  Suero fisiológico 5 ml   2 lote(s)   65 uds.
  Apósitos estériles       1 lote(s)   50 uds.
Total: 3 medicamentos, 4 lotes, 145 unidades
Caducan en los próximos 30 días:
  ⚠️ L2503 (Suero fisiológico 5 ml) caduca el 2026-10-31
```

`datos/botiquin_revisado.xml`:

```xml
<?xml version="1.0" encoding="UTF-8" standalone="no"?>
<botiquin centro="CIFP Avilés">
    <medicamento codigo="M01" nombre="Paracetamol 500 mg">
        <lote caducidad="2027-05-31" numero="L2405">30</lote>
    </medicamento>
    <medicamento codigo="M03" nombre="Suero fisiológico 5 ml">
        <lote caducidad="2026-10-31" numero="L2503">40</lote>
        <lote caducidad="2028-01-31" numero="L2602">25</lote>
    </medicamento>
    <medicamento codigo="M04" nombre="Apósitos estériles">
        <lote caducidad="2029-12-31" numero="L2412">50</lote>
    </medicamento>
</botiquin>
```

> **Notas para el aula:** comprobado con la versión «ingenua» (bucle de lotes hacia delante): retira solo 3 lotes, porque al borrar L2501 la lista viva desplaza L2502 a su posición y el `j++` se lo salta; L2502 (caducado) quedaría en el fichero y el resumen SAX lo contaría. Por la misma razón los medicamentos vacíos se borran después del recorrido. Las fechas ISO (`aaaa-mm-dd`) se podrían comparar como texto con `compareTo`, pero `LocalDate` deja claro qué se compara y valida el formato. En SAX, los atributos solo están disponibles en `startElement()`: por eso se guardan en campos del *handler* para usarlos en `endElement()`.